

Group project - Wine - Jared Martin - R Code

Jared Martin

2024-11-24

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(class)
library(tree)
library(randomForest)
```

```
## randomForest 4.7-1.1
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##   combine
##
## The following object is masked from 'package:ggplot2':
##
##   margin
```

```
library(e1071)
library(nnet)
```

Group Members: 1 of 1 Jared Martin

Cleaning and Preparing the data

The following code pulls in the two wine data sets, creates a new variable called wine type (red or white), makes quality and type factor variables, joins the two data sets together and saves the data as a csv.

```

rbind(read.csv("./Data/winequality-red.csv", sep = ";") %>%
      mutate(wine.type = "red"),
      read.csv("./Data/winequality-red.csv", sep = ";") %>%
      mutate(wine.type = "white")) %>%
write.csv( "./Data/wine_data.csv")

wine_data <- read_csv("./Data/wine_data.csv")

## New names:
## Rows: 3198 Columns: 14
## -- Column specification
## ----- Delimiter: "," chr
## (1): wine.type dbl (13): ...1, fixed.acidity, volatile.acidity, citric.acid,
## residual.sugar...
## i Use 'spec()' to retrieve the full column specification for this data. i
## Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## * ' -> '...1'

wine_data$quality <- factor(wine_data$quality)

wine_data$wine.type <- factor(wine_data$wine.type)

wine_data[,2:14] -> wine_data

wine_data %>%
  dplyr::select(quality, everything()) -> wine_data

```

Research Question #1: Can we predict the quality of a wine given its characteristic?

Each wine was given as ranking for quality between 0-10. Although in the data set the lowest level of quality is 3, while the highest is an 8. Given that there is more than two level of quality we will be using KKN and classification trees to predict the quality of the wine given its characteristics.

```
levels(wine_data$quality)
```

```
## [1] "3" "4" "5" "6" "7" "8"
```

Method #1: KNN

1. Splitting the data into a testing and training dataset.

```

set.seed(1)

n <- nrow(wine_data)

z <- sample(n, n/2)

```

```

wine_test <- wine_data[z,]

wine_train <- wine_data[-z,]

x.train <- wine_train[,2:12]
x.test <- wine_test[,2:12]

y.train <- wine_train$quality
y.test <- wine_test$quality

dim(x.train)[1] == length(y.train) # Test to see if length is the same. Should be TRUE

## [1] TRUE

dim(x.test)[1] == length(y.test) # Test to see if length is the same. Should be TRUE

## [1] TRUE

```

2. Determining the optimal number of nearest neighbors “K”

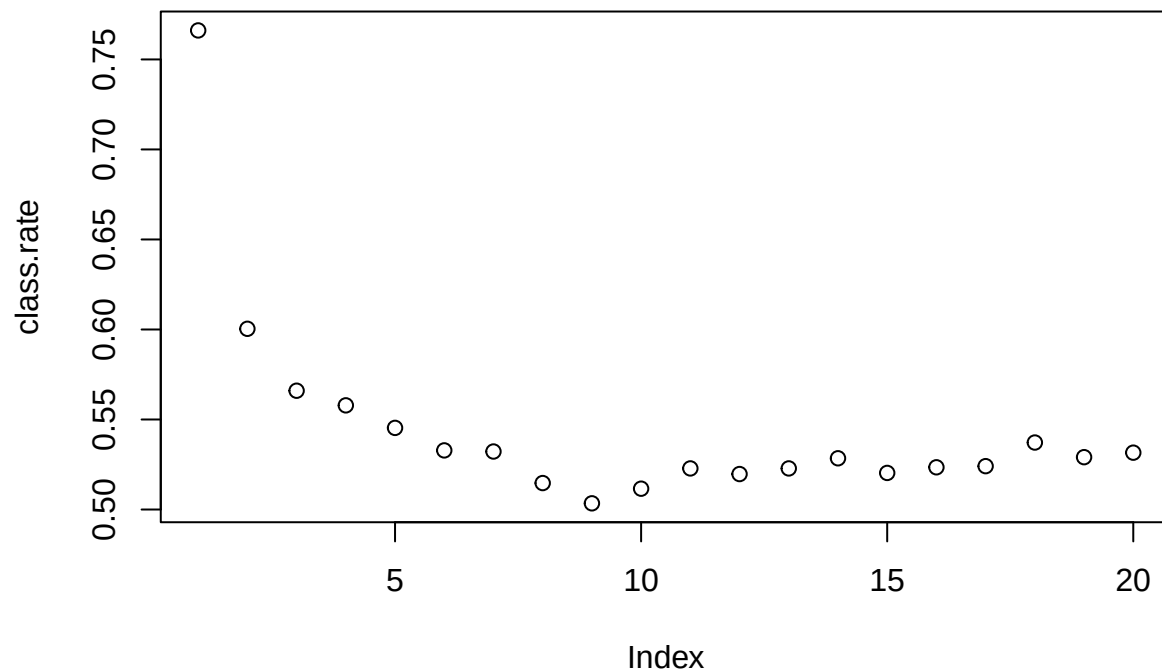
```

class.rate = rep(0,20)

for (K in 1:20) {
  knn.result = knn(x.train, x.test, y.train, K)
  class.rate[K] = mean( y.test == knn.result )
}

plot(class.rate)

```



```
max(class.rate) # optimal number of K neighbors is 1
```

```
## [1] 0.7661038
```

3. Cross Validation - Classification Rate and Confusion table using Optimal number of nearest Neighbors.

```
knn_cv_wine_quality <- knn(x.train, x.test, y.train, 1)
```

```
table(y.test, knn_cv_wine_quality)
```

```
##      knn_cv_wine_quality
## y.test  3  4  5  6  7  8
##      3  7  4  2  0  0  0
##      4  0 30 12 10  2  0
##      5  2  8 541 90 16  0
##      6  2 12 116 501 18  6
##      7  0  2  20  40 138  0
##      8  0  0  0  6  6  8
```

```
knn_cv_classification_rate <- mean(y.test == knn_cv_wine_quality)
```

```
# Classification rate of .766
```

4. LOOCV - Classification Rate and Confusion table using Optimal number of nearest Neighbors.

```
y.test <- wine_data$quality
```

```
x.train <- wine_data[,2:12]
```

```
knn_loocv_wine_quality <- knn.cv(x.train, y.test, k = 1, prob = FALSE)
```

```
table(y.test, knn_loocv_wine_quality)
```

```
##      knn_loocv_wine_quality
## y.test  3  4  5  6  7  8
##      3 20  0  0  0  0  0
##      4  0 106  0  0  0  0
##      5  0  0 1362  0  0  0
##      6  0  0  0 1276  0  0
##      7  0  0  0  0 398  0
##      8  0  0  0  0  0 36
```

```
knn_loocv_classification_rate <- mean(y.test == knn_loocv_wine_quality)
```

```
#Wow! Classification rate of 1!
```

Method #2: Classification Trees

1. Classification Tree for Quality

```
tree_wine_quality <- tree(quality ~ ., data = wine_train)
```

```
tree_wine_quality
```

```
## node), split, n, deviance, yval, (yprob)
##      * denotes terminal node
##
##  1) root 1599 3736.00 5 ( 0.004378 0.032520 0.440901 0.388368 0.123827 0.010006 )
##    2) alcohol < 10.525 996 1887.00 5 ( 0.006024 0.032129 0.601406 0.322289 0.035141 0.003012 )
##      4) sulphates < 0.615 566 898.10 5 ( 0.005300 0.044170 0.687279 0.259717 0.003534 0.000000 )
##        8) total.sulfur.dioxide < 98.5 479 818.40 5 ( 0.006263 0.052192 0.640919 0.296451 0.004175 0.000000 )
##          16) sulphates < 0.525 178 278.10 5 ( 0.005618 0.095506 0.747191 0.146067 0.005618 0.000000 )
##            17) sulphates > 0.525 301 501.50 5 ( 0.006645 0.026578 0.578073 0.385382 0.003322 0.000000 )
##              9) total.sulfur.dioxide > 98.5 87 38.27 5 ( 0.000000 0.000000 0.942529 0.057471 0.000000 0.000000 )
##                5) sulphates > 0.615 430 902.50 5 ( 0.006977 0.016279 0.488372 0.404651 0.076744 0.006977 )
##                  10) alcohol < 9.85 243 419.40 5 ( 0.004115 0.016461 0.625514 0.325103 0.024691 0.004115 )
##                    20) fixed.acidity < 10.75 220 340.60 5 ( 0.004545 0.018182 0.681818 0.281818 0.013636 0.000000 )
##                      21) fixed.acidity > 10.75 23 38.54 6 ( 0.000000 0.000000 0.086957 0.739130 0.130435 0.043478 )
##                        11) alcohol > 9.85 187 430.10 6 ( 0.010695 0.016043 0.310160 0.508021 0.144385 0.010695 ) *
##                          3) alcohol > 10.525 603 1463.00 6 ( 0.001658 0.033167 0.175788 0.497512 0.270315 0.021559 )
##                            6) volatile.acidity < 0.385 222 488.90 7 ( 0.000000 0.009009 0.090090 0.396396 0.468468 0.036036 )
##                              7) volatile.acidity > 0.385 381 889.80 6 ( 0.002625 0.047244 0.225722 0.556430 0.154856 0.013113 )
##                                14) alcohol < 11.45 208 428.60 6 ( 0.004808 0.062500 0.298077 0.581731 0.052885 0.000000 ) *
##                                  15) alcohol > 11.45 173 405.70 6 ( 0.000000 0.028902 0.138728 0.526012 0.277457 0.028902 )
##                                    30) sulphates < 0.635 86 183.10 6 ( 0.000000 0.058140 0.232558 0.593023 0.116279 0.000000 )
##                                      31) sulphates > 0.635 87 178.30 6 ( 0.000000 0.000000 0.045977 0.459770 0.436782 0.057471 )
```

```
summary(tree_wine_quality)
```

```
##
## Classification tree:
## tree(formula = quality ~ ., data = wine_train)
## Variables actually used in tree construction:
## [1] "alcohol"          "sulphates"        "total.sulfur.dioxide"
## [4] "fixed.acidity"    "volatile.acidity"
## Number of terminal nodes: 10
## Residual mean deviance: 1.829 = 2906 / 1589
## Misclassification error rate: 0.3952 = 632 / 1599
```

not all class are represented as terminal node classifications (3, 4 & 8 are not represented).

2. Calculating the Classification Rate for Unpruned Tree

```
tree_hat <- predict(tree_wine_quality, wine_test, type = "class")
```

```
table(tree_hat, wine_test$quality)
```

```
##
## tree_hat   3   4   5   6   7   8
##           3   0   0   0   0   0   0
##           4   0   0   0   0   0   0
##           5   6  35 483 233   9   0
##           6   7  19 148 322 103  12
##           7   0   0  26 100  88   8
##           8   0   0   0   0   0   0
```

```
tree_classification_rate <- mean(tree_hat == wine_test$quality)
```

3. Pruning the Classification tree on misclassification rate.

The optimal number of terminal nodes on a tree is 8. The same number of terminal nodes as the tree above

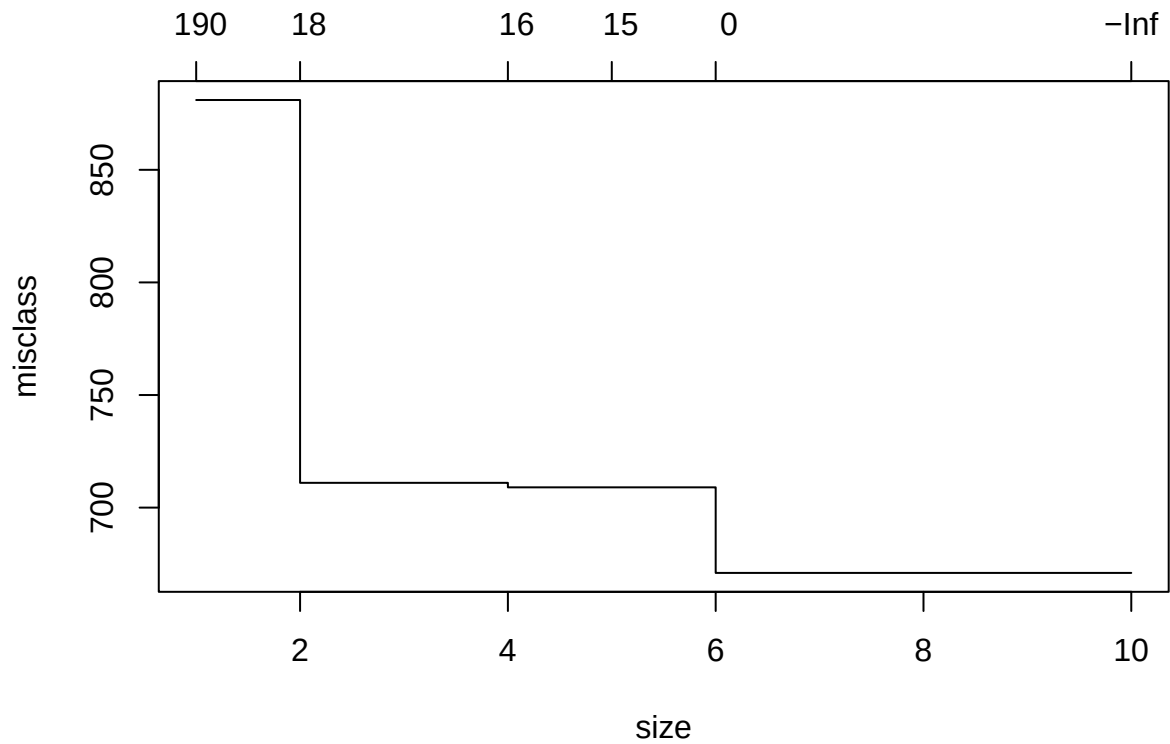
```
set.seed(1)
```

```
tree_cv_wine_quality <- cv.tree( tree_wine_quality, FUN = prune.misclass )
```

```
tree_cv_wine_quality
```

```
## $size
## [1] 10  6  5  4  2  1
##
## $dev
## [1] 671 671 709 709 711 881
##
## $k
## [1] -Inf  0.0 15.0 16.0 18.5 194.0
##
## $method
## [1] "misclass"
##
## attr("class")
## [1] "prune"          "tree.sequence"
```

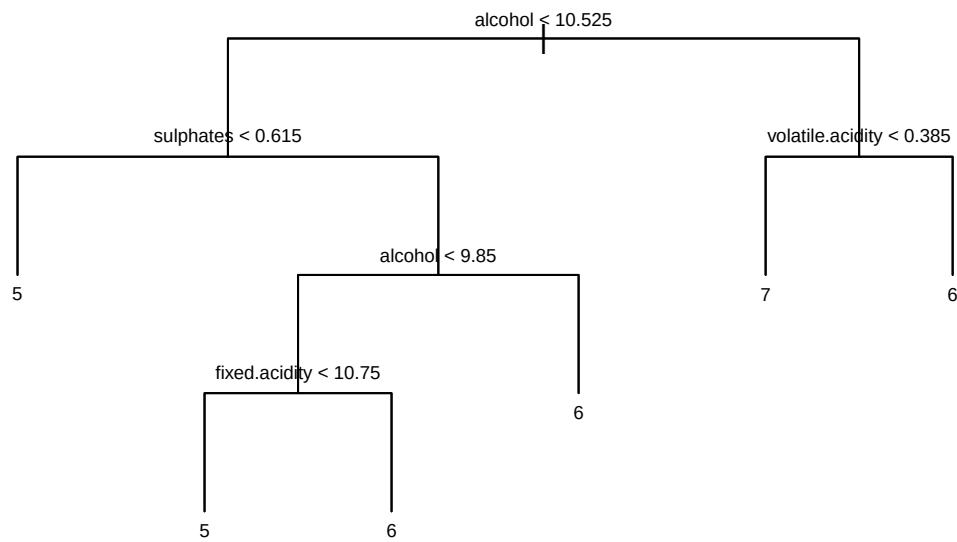
```
plot(tree_cv_wine_quality)
```



```
tree_pruned_wine_quality <- prune.misclass(tree_wine_quality, best = 6)
```

4. Plotting Classification Tree

```
plot(tree_pruned_wine_quality, type = "uniform")
text(tree_pruned_wine_quality, cex = 0.6)
```



of the Classification Tree

5. Cross Validation

```
tree_hat <- predict(tree_pruned_wine_quality, wine_test, type = "class")

table(tree_hat, wine_test$quality)
```

```
##
## tree_hat   3   4   5   6   7   8
##          3   0   0   0   0   0
##          4   0   0   0   0   0
##          5   6  35 483 233   9   0
##          6   7  19 148 322 103  12
##          7   0   0  26 100  88   8
##          8   0   0   0   0   0   0
```

```
tree_pruned_classification_rate <- mean(tree_hat == wine_test$quality)
```

##Method #3: Random Forest

1. Creating the Training Random Forest

```
set.seed(1)

forest_wine_quality <- randomForest(quality ~ ., data = wine_train)

forest_wine_quality
```

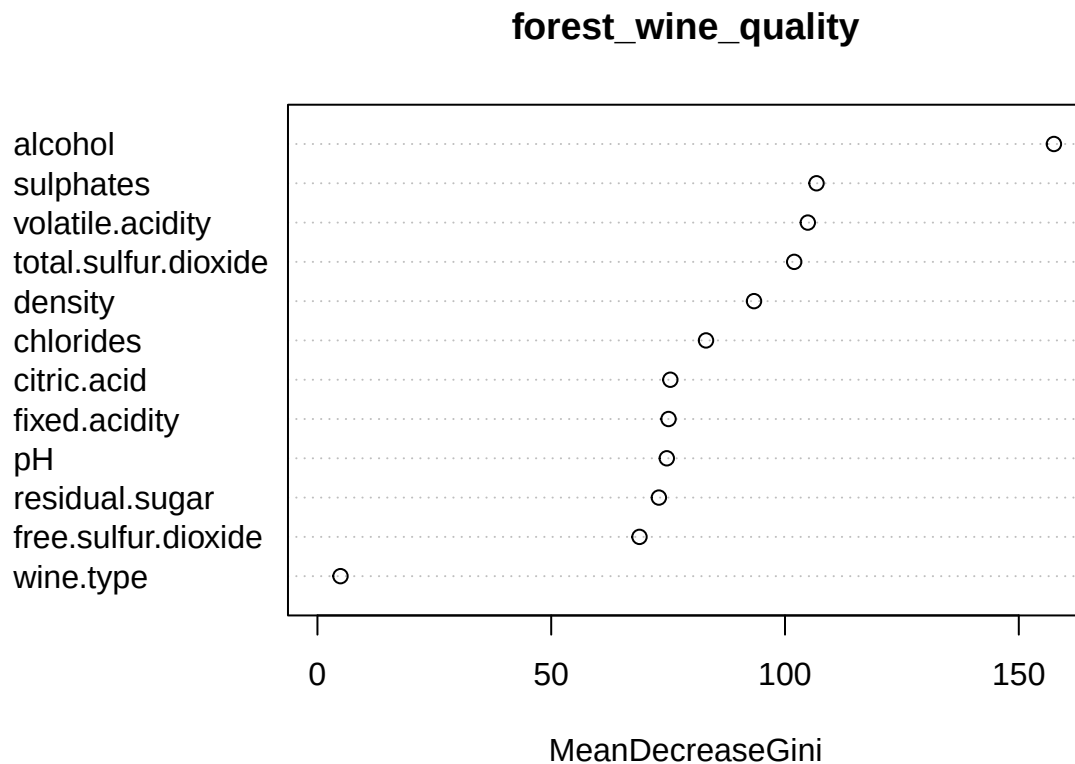
```
##
## Call:
## randomForest(formula = quality ~ ., data = wine_train)
##           Type of random forest: classification
##           Number of trees: 500
## No. of variables tried at each split: 3
##
##           OOB estimate of  error rate: 15.88%
## Confusion matrix:
##   3  4  5  6  7  8 class.error
## 3 2  2  2  1  0  0  0.7142857
## 4 0 22 23  7  0  0  0.5769231
## 5 0  0 632 70  3  0  0.1035461
## 6 0  0 76 525 20  0  0.1545894
## 7 0  0  4 39 155 0  0.2171717
## 8 0  0  0  5  2  9  0.4375000
```

```
importance(forest_wine_quality)
```

```
##
##           MeanDecreaseGini
## fixed.acidity           75.092489
## volatile.acidity       104.878037
## citric.acid             75.473790
## residual.sugar          73.024754
```

```
## chlorides            83.108985
## free.sulfur.dioxide  68.874209
## total.sulfur.dioxide 101.966127
## density             93.371748
## pH                  74.719650
## sulphates           106.741168
## alcohol             157.514464
## wine.type           4.932927
```

```
varImpPlot(forest_wine_quality)
```



2.

Searching for the optimal number of trees and number of variables considered at each branch.

```
set.seed(1)

err.rate <- numeric(10) # To store error rates for each mtry
n.trees <- numeric(10)  # To store the number of optimal trees for each mtry

for (m in 1:10) {
  forest_wine_quality_m <- randomForest(quality ~ ., data = wine_train,
                                         mtry = m )
  opt.tree <- which.min(forest_wine_quality_m$err.rate)
  forest_wine_quality_m <- randomForest(quality ~ ., data = wine_train,
                                         mtry = m, ntree = opt.tree)
  yhat_rf_m <- predict(forest_wine_quality_m, newdata = wine_test, type = "class")
  err.rate[m] <- 1 - mean(wine_test$quality == yhat_rf_m)
  n.trees[m] <- opt.tree
}

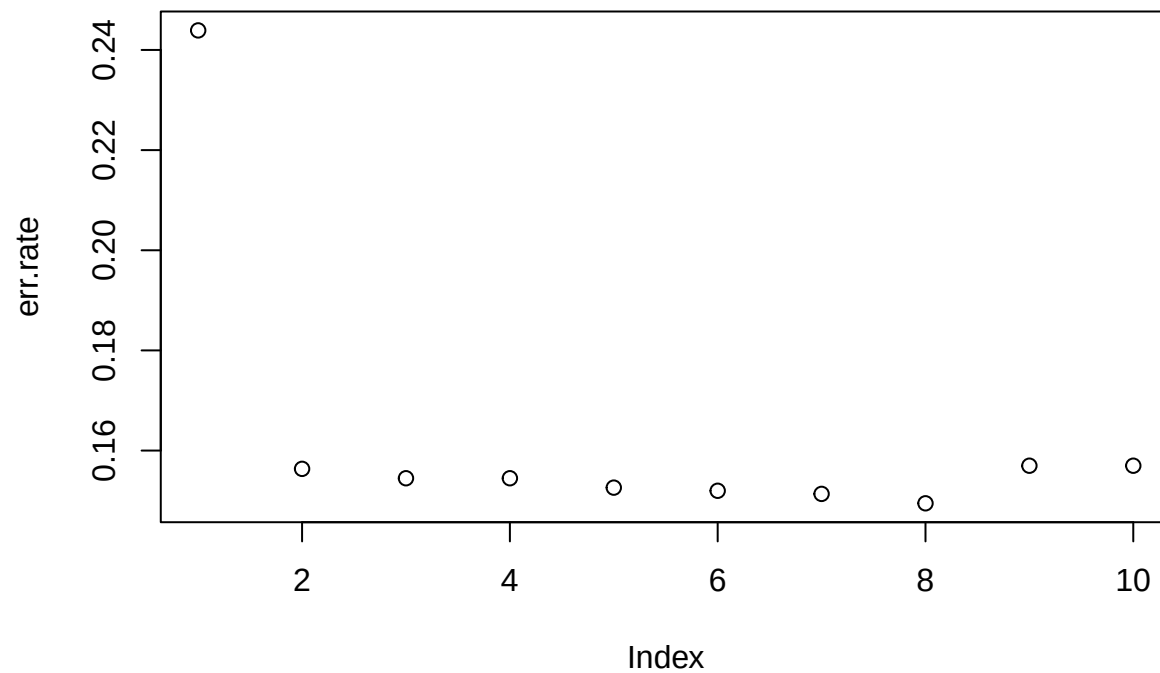
err.rate
```

```
## [1] 0.2439024 0.1563477 0.1544715 0.1544715 0.1525954 0.1519700 0.1513446
## [8] 0.1494684 0.1569731 0.1569731
```

```
n.trees
```

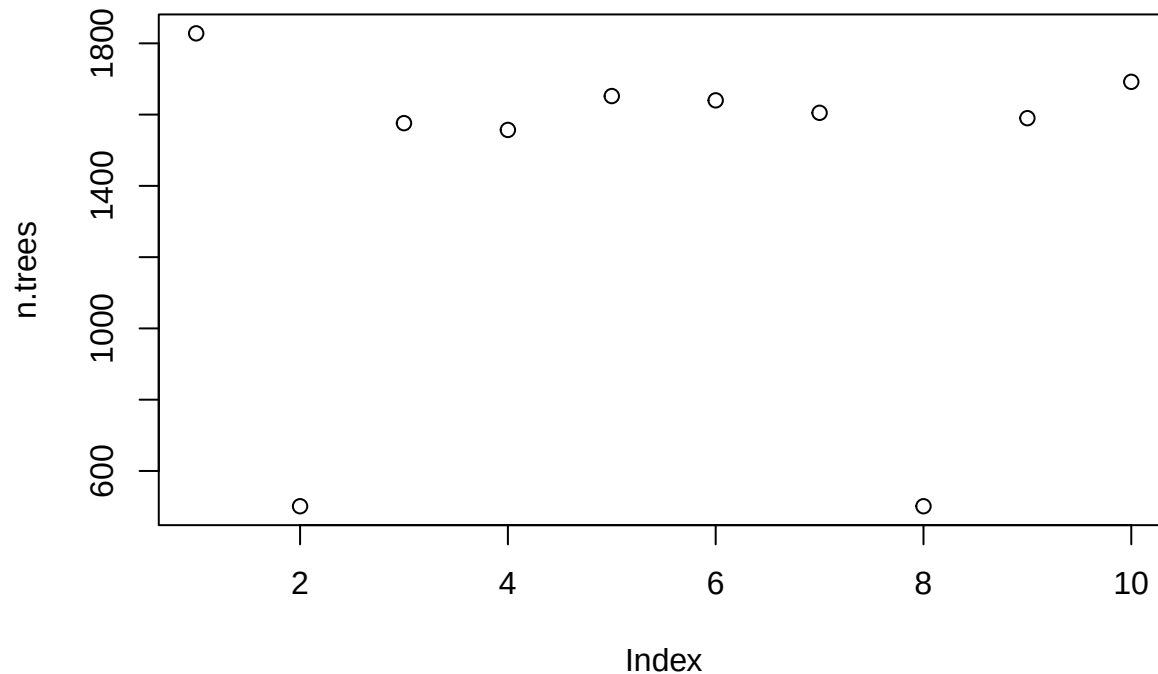
```
## [1] 1828 501 1576 1557 1652 1640 1605 501 1590 1692
```

```
plot(err.rate); line(err.rate)
```



```
##
## Call:
## line(err.rate)
##
## Coefficients:
## [1] 0.1552012 -0.0002085
```

```
plot(n.trees); line(n.trees)
```



```
##
## Call:
## line(n.trees)
##
## Coefficients:
## [1] 1564.667 5.167
```

optimal solution is 501 trees and 8 variables considered at each branch.

3. Cross Validation with Optimal number of trees and number of variables considered at each branch.

```
set.seed(1)

forest_cv_wine_quality <- randomForest(quality ~ ., data = wine_train,
                                       mtry = 8, ntree = 501 )

yhat_rf <- predict(forest_cv_wine_quality, newdata = wine_test, type = "class")

forest_classification_rate <- mean(wine_test$quality == yhat_rf)
```

Method #4: Support Vecot Machines

1. Creating a SVM model with a linear bounder

```
SVM_wine_quality <- svm(quality ~ ., data = wine_train, kernel="linear")

summary(SVM_wine_quality)
```

```
##
## Call:
## svm(formula = quality ~ ., data = wine_train, kernel = "linear")
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: linear
##         cost: 1
##
## Number of Support Vectors: 1356
##
## ( 481 602 198 52 16 7 )
##
##
## Number of Classes: 6
##
## Levels:
## 3 4 5 6 7 8
```

2. Tuning the SVM model to determine the optimal kernel and cost.

```
set.seed(1)

svm_tuned <- tune(
  svm,
  quality ~ .,
  data = wine_train,
  ranges = list(
    cost = 10^seq(-2, 2, by = 1),
    kernel = c("linear", "polynomial", "radial", "sigmoid") # Test multiple kernels
  )
)
```

```
summary(svm_tuned)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##   cost kernel
##   100 radial
##
## - best performance: 0.3283097
##
## - Detailed performance results:
```

	cost	kernel	error	dispersion
## 1	1e-02	linear	0.4258451	0.04861307
## 2	1e-01	linear	0.4120755	0.04892215
## 3	1e+00	linear	0.4127005	0.05053833

```
## 4 1e+01 linear 0.4133255 0.05057421
## 5 1e+02 linear 0.4108255 0.05097838
## 6 1e-02 polynomial 0.5540566 0.04555867
## 7 1e-01 polynomial 0.4583333 0.05584991
## 8 1e+00 polynomial 0.3683176 0.03124191
## 9 1e+01 polynomial 0.3439662 0.02748956
## 10 1e+02 polynomial 0.3533412 0.03087061
## 11 1e-02 radial 0.5590605 0.04940126
## 12 1e-01 radial 0.4089662 0.04089455
## 13 1e+00 radial 0.3664387 0.03796483
## 14 1e+01 radial 0.3301965 0.02550939
## 15 1e+02 radial 0.3283097 0.03640336
## 16 1e-02 sigmoid 0.4783451 0.06421844
## 17 1e-01 sigmoid 0.4245873 0.05193990
## 18 1e+00 sigmoid 0.4877948 0.02711297
## 19 1e+01 sigmoid 0.5146816 0.03254922
## 20 1e+02 sigmoid 0.5053027 0.03842214
```

```
# optimal cost is 10 with a radial boundary.
```

3. Cross Validation with Optimal Cost and Kernel

```
SVM_wine_quality_optimal <- svm(quality ~ .,
                                data = wine_train,
                                kernel="radial",
                                cost = 100)

summary(SVM_wine_quality_optimal)
```

```
##
## Call:
## svm(formula = quality ~ ., data = wine_train, kernel = "radial",
##      cost = 100)
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: radial
##      cost:  100
##
## Number of Support Vectors:  1165
##
## ( 431 492 167 52 16 7 )
##
##
## Number of Classes:  6
##
## Levels:
##  3 4 5 6 7 8
```

```
svm_yhat <- predict(SVM_wine_quality_optimal, data = wine_test)

table(svm_yhat, wine_test$quality)
```

```
##
## svm_yhat   3   4   5   6   7   8
##          3   0   1   2   3   1   0
##          4   0   1  20  27   1   0
##          5   7  22 302 305  94  10
##          6   5  28 245 245  71   8
##          7   1   2  80  71  31   2
##          8   0   0   8   4   2   0
```

```
svm_cv_classification_rate <- mean(svm_yhat == wine_test$quality)
```

Method #5: Artificial Neural Network

1. Searching for the optimal size of hidden nodes

```
set.seed(1)

err.rate <- numeric(10)

for (m in 1:12) {
  ann_wine_quality <- nnet(quality ~ .,
                           data = wine_train,
                           size = m)
  yhat_ann <- predict(ann_wine_quality, newdata = wine_test, type = "class")
  err.rate[m] <- 1 - mean(wine_test$quality == yhat_ann)
}
```

```
## # weights:  25
## initial  value 2417.450223
## iter  10 value 1869.236229
## iter  20 value 1863.382746
## iter  30 value 1861.552110
## final   value 1861.549414
## converged
## # weights:  44
## initial  value 3275.930458
## iter  10 value 1849.847362
## iter  20 value 1791.876795
## iter  30 value 1782.937612
## iter  40 value 1734.692249
## iter  50 value 1610.752711
## iter  60 value 1540.956955
## iter  70 value 1508.026206
## iter  80 value 1484.361300
## iter  90 value 1475.422685
## iter 100 value 1462.063732
```

```

## final value 1462.063732
## stopped after 100 iterations
## # weights: 63
## initial value 3649.455939
## iter 10 value 1794.835512
## iter 20 value 1755.507824
## iter 30 value 1587.077809
## iter 40 value 1508.257590
## iter 50 value 1474.341888
## iter 60 value 1464.051839
## iter 70 value 1455.651657
## iter 80 value 1452.101496
## iter 90 value 1449.652595
## iter 100 value 1448.565825
## final value 1448.565825
## stopped after 100 iterations
## # weights: 82
## initial value 4145.198519
## iter 10 value 1896.018664
## iter 20 value 1836.555432
## iter 30 value 1769.828575
## iter 40 value 1660.613648
## iter 50 value 1570.374740
## iter 60 value 1495.587389
## iter 70 value 1478.661985
## iter 80 value 1474.924473
## iter 90 value 1472.074010
## iter 100 value 1462.620804
## final value 1462.620804
## stopped after 100 iterations
## # weights: 101
## initial value 3206.080529
## iter 10 value 1868.885265
## final value 1868.126494
## converged
## # weights: 120
## initial value 3904.080130
## iter 10 value 1842.679882
## iter 20 value 1774.095023
## iter 30 value 1633.863611
## iter 40 value 1501.436412
## iter 50 value 1463.361404
## iter 60 value 1453.327482
## iter 70 value 1448.772394
## iter 80 value 1445.856669
## iter 90 value 1439.586327
## iter 100 value 1425.486901
## final value 1425.486901
## stopped after 100 iterations
## # weights: 139
## initial value 3003.376554
## iter 10 value 1819.382482
## iter 20 value 1730.399260
## iter 30 value 1598.996707

```

```

## iter 40 value 1480.987103
## iter 50 value 1459.040641
## iter 60 value 1451.882566
## iter 70 value 1447.272097
## iter 80 value 1446.098132
## iter 90 value 1444.158753
## iter 100 value 1443.200769
## final value 1443.200769
## stopped after 100 iterations
## # weights: 158
## initial value 3631.981841
## iter 10 value 1883.299565
## iter 20 value 1762.948030
## iter 30 value 1582.185471
## iter 40 value 1542.134189
## iter 50 value 1480.197939
## iter 60 value 1456.141032
## iter 70 value 1445.450353
## iter 80 value 1441.505310
## iter 90 value 1439.464096
## iter 100 value 1438.344936
## final value 1438.344936
## stopped after 100 iterations
## # weights: 177
## initial value 2829.842145
## iter 10 value 1795.548105
## iter 20 value 1753.356955
## iter 30 value 1668.471819
## iter 40 value 1581.819159
## iter 50 value 1496.770509
## iter 60 value 1442.762498
## iter 70 value 1405.066969
## iter 80 value 1400.581356
## iter 90 value 1391.891404
## iter 100 value 1382.325810
## final value 1382.325810
## stopped after 100 iterations
## # weights: 196
## initial value 3490.707771
## iter 10 value 1865.413747
## iter 20 value 1794.742400
## iter 30 value 1703.104722
## iter 40 value 1534.019412
## iter 50 value 1437.998610
## iter 60 value 1417.852096
## iter 70 value 1394.893530
## iter 80 value 1386.554254
## iter 90 value 1373.670488
## iter 100 value 1360.839037
## final value 1360.839037
## stopped after 100 iterations
## # weights: 215
## initial value 2727.946619
## iter 10 value 1888.876300

```

```

## iter 20 value 1777.537358
## iter 30 value 1629.496179
## iter 40 value 1527.498274
## iter 50 value 1468.471286
## iter 60 value 1434.159507
## iter 70 value 1405.258105
## iter 80 value 1367.291281
## iter 90 value 1342.331223
## iter 100 value 1323.336084
## final value 1323.336084
## stopped after 100 iterations
## # weights: 234
## initial value 3816.500485
## iter 10 value 1839.561582
## iter 20 value 1761.989451
## iter 30 value 1691.518295
## iter 40 value 1575.603798
## iter 50 value 1479.662615
## iter 60 value 1426.919300
## iter 70 value 1403.739839
## iter 80 value 1400.557743
## iter 90 value 1391.712918
## iter 100 value 1380.774611
## final value 1380.774611
## stopped after 100 iterations

```

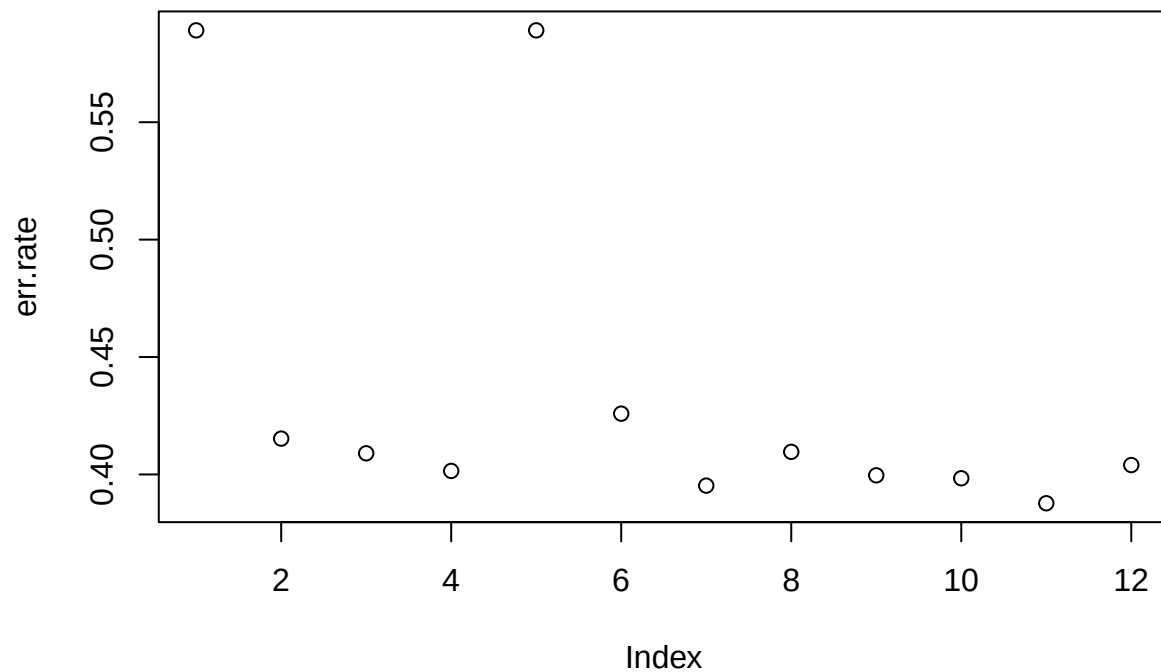
```
err.rate
```

```

## [1] 0.5891182 0.4152595 0.4090056 0.4015009 0.5891182 0.4258912 0.3952470
## [8] 0.4096310 0.3996248 0.3983740 0.3877423 0.4040025

```

```
plot(err.rate)
```



```
# 11 hidden nodes produce the lowest prediction error rate
```

2. Cross Validation with Optimal number of hidden nodes

```
set.seed(1)

ann_wine_quality_optimal <- nnet(quality ~ .,
                                data = wine_train,
                                size = 11)

## # weights:  215
## initial  value 2420.843849
## iter   10 value 1803.434061
## iter   20 value 1682.911449
## iter   30 value 1526.889996
## iter   40 value 1453.751404
## iter   50 value 1426.973835
## iter   60 value 1408.019425
## iter   70 value 1406.208155
## iter   80 value 1399.941861
## iter   90 value 1392.897539
## iter  100 value 1385.694391
## final   value 1385.694391
## stopped after 100 iterations

yhat_ann <- predict(ann_wine_quality, newdata = wine_test, type = "class")

ann_classification_rate <- mean(wine_test$quality == yhat_ann)
```

Summary: Which model performed at predicting the quality of a wine given its characteristics?

Using cross validation evaluating the performance of each model, the random forest model with 501 trees and 8 mtry produced the lowest error rate of 15.2%.

```
classification_rates <- data.frame(
  Model = c("K-Nearest Neighbors", "Classification Tree", "Random Forest", "Support Vector Machines", "ANN"),
  Classification_Rate = c(knn_cv_classification_rate,
                         tree_pruned_classification_rate,
                         forest_classification_rate,
                         svm_cv_classification_rate,
                         ann_classification_rate),
  Error_Rate = c(1 - knn_cv_classification_rate,
                 1 - tree_pruned_classification_rate,
                 1 - forest_classification_rate,
                 1 - svm_cv_classification_rate,
                 1 - ann_classification_rate)
)
```

```
classification_rates %>%  
  arrange(desc(Classification_Rate))
```

##	Model	Classification_Rate	Error_Rate
## 1	Random Forest	0.8480300	0.1519700
## 2	K-Nearest Neighbors	0.7661038	0.2338962
## 3	Artifical Nerual Network	0.5959975	0.4040025
## 4	Classifcation Tree	0.5584740	0.4415260
## 5	Support Vector Machines	0.3621013	0.6378987